Disclaimer

This document is an exam summary and follows the given material of the lecture *Deep Learning*. Its contribution is a short summary that contains the most important concepts, formulas and algorithms. Due to curriculum content updates, some content may not be relevant to future versions of the course.

I do not guarantee the accuracy or completeness, nor is this document endorsed by the instructors. Any errors that are pointed out to me are welcome. The complete LaTeX source code can be found at `https://github.com/tstreule/eth-cheat-sheets`.

Deep Learning

Timo Streule, tstreule@ethz.ch (02.02.2022)

1. Connectionism

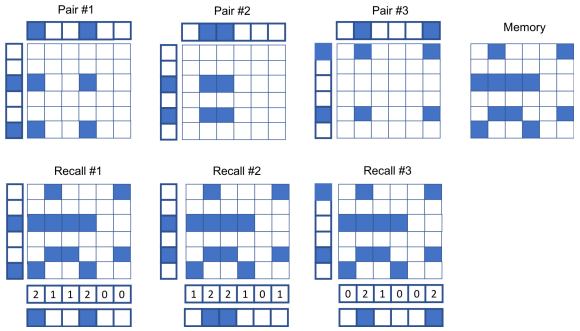
1.1. Willshaw memory

Associative memory, pairs of $(\mathbf{x}^t, \mathbf{y}^t) \in \mathcal{S}$. Learn $\mathbf{x}^t \mapsto \mathbf{y}^t$:

W. learning: $\begin{cases} \Theta_{ji} = \min\{1, \sum_{t=1}^s y_j^t x_i^t\} \\ \Theta^t = \mathbf{y}^t(\mathbf{x}^t)^\top, \quad \Theta = \min\{\mathbf{1}, \sum_{t=1}^s \Theta^t\} \end{cases}$

Since $\Theta^t \mathbf{x}^t = (\mathbf{x}^t \cdot \mathbf{x}^t)\mathbf{y}^t = r\mathbf{y}^t$, retrieving y for a given $\mathbf{x}$ is:

W. retrieval: $\mathbf{x} \mapsto \mathbf{z} \mapsto \mathbf{y}$, $\mathbf{z} = \Theta\mathbf{x}$, $y_j = \begin{cases} 0 & z_j < r \\ 1 & \text{otw.} \end{cases}$



1.2. Hopfield network

Assume neurons $x_i \in \{-1, 1\}$ with couplings $\Theta = (\theta)_{ij}$ such that

(1) symmetric: $\theta_{ij} = \theta_{ji}$ (2) irreflexive: $\theta_{ii} = 0$.

The **update scheme** is given as follows

H. learning: $x_i \leftarrow \text{sign}\{\sum_{j \neq i} \theta_{ij} x_j + \theta_{i0}\}$ (bias)

Convergence is apparent from the Lyapunov equation

Ising model: $\mathcal{E} = -\sum_{i,j=1}^n \theta_{ij} x_i x_j - \sum_{i=1}^n \theta_{i0} x_i$

Recurr. netw. that can work as **auto-associative** memory:

H. memory: $\Theta \propto \sum_{t=1}^s [\mathbf{x}_t \mathbf{x}_t^\top - \mathbf{I}] \in \mathbb{Z}^{n \times n}$

2. Linear Networks

2.1. Linear Units

A linear unit defines a unique direction of change via $\boldsymbol{\theta}/\|\boldsymbol{\theta}\|$, as well as a constant ☹ rate of change via $\|\boldsymbol{\theta}\|$.

Linear, affine unit: $\iota(\tilde{\mathbf{x}}, \boldsymbol{\theta}) = \tilde{\mathbf{x}}^\top \boldsymbol{\theta} \equiv \sum_{i=1}^n x_i \theta_i + b$

Multiple linear units form a **layer**, resulting in

Linear map: $L(\mathbf{x}; \boldsymbol{\Theta}) = \begin{bmatrix} \cdots \\ \mathbf{x}^\top \boldsymbol{\theta}^{(j)} \\ \cdots \end{bmatrix} = \boldsymbol{\Theta}\mathbf{x}, \quad \boldsymbol{\Theta} \in \mathbb{R}^{m \times n}$

Simple parameter update rule

Delta rule: $\begin{cases} \Delta\boldsymbol{\theta} = \eta(y - \mathbf{x}^\top \boldsymbol{\theta})\mathbf{x} \\ \Delta\boldsymbol{\Theta} = \eta(\mathbf{y} - \boldsymbol{\Theta}\mathbf{x})\mathbf{x}^\top \end{cases}$ as simple parameter update rule.

2.2. Linearity

A func. f is **linear / affine** $\iff f(\alpha\mathbf{x} + \beta\mathbf{y}) = \alpha f(\mathbf{x}) + \beta f(\mathbf{y}) + \mathbf{b}$

$\iff f(\mathbf{x}) = \mathbf{A}\mathbf{x} \quad (\forall \mathbf{x}, \mathbf{y} \in \mathbb{R}^n,\ \forall \alpha, \beta \in \mathbb{R},\ \mathbf{A} \in \mathbb{R}^{n \times m},\ \mathbf{b} \in \mathbb{R}^m)$

Composition: $f(\mathbf{x}) = \mathbf{A}\mathbf{x},\ g(\mathbf{x}) = \mathbf{B}\mathbf{x} \implies (g \circ f)(x) = \mathbf{B}\mathbf{A}\mathbf{x}$

2.3. Linear Autoencoder

Linear AE: $\begin{cases} \mathbf{x} \overset{\mathbf{C}}{\longmapsto} \mathbf{z} \overset{\mathbf{D}}{\longmapsto} \hat{\mathbf{x}}, \quad \mathbf{C}, \mathbf{D}^\top \in \mathbb{R}^{m \times n},\ m < n \\ \ell(\mathbf{x}) = \frac{1}{2}\|\mathbf{x} - \hat{\mathbf{x}}\|^2 = \frac{1}{2}\|\mathbf{x} - \mathbf{D}\mathbf{C}\mathbf{x}\|^2 \end{cases}$

Auto-loss: $(\mathbf{C}^*, \mathbf{D}^*) \xleftarrow{\arg\min} \frac{1}{2s}\sum_{t=1}^s \ell(\mathbf{x}_t) = \frac{1}{2}\|\mathbf{X} - \mathbf{D}\mathbf{C}\mathbf{X}\|_F^2$

with **Frobenius norm** $\|\mathbf{A}\|_F := \sqrt{\sum_{i,j} |a_{ij}|^2}$

The **goal** is to learn the identity $\mathbf{D}\mathbf{C} \approx \mathbf{I}$ via the **AE gradients**

$\frac{\partial \ell(\mathbf{x})}{\partial \mathbf{C}} = \mathbf{D}^\top(\hat{\mathbf{x}} - \mathbf{x})\mathbf{x}^\top \in \mathbb{R}^{m \times n}$, $\frac{\partial \ell(\mathbf{x})}{\partial \mathbf{D}} = (\hat{\mathbf{x}} - \mathbf{x})\mathbf{x}^\top \mathbf{C}^\top \in \mathbb{R}^{n \times m}$

Eckard-Young: Choose $\mathbf{D} = \mathbf{U}_k$ and $\mathbf{C} = \mathbf{U}_k^\top$ given $\mathbf{X} = \mathbf{U}\boldsymbol{\Sigma}\mathbf{V}^\top$ (not unique!)

Reduced rank: $\text{rank}(\mathbf{D}\mathbf{C}) \leq \min\{\text{rank}(\mathbf{C}), \text{rank}(\mathbf{D})\} \leq m < n$

$\widehat{\mathbf{X}} = \mathbf{D}\mathbf{C}\mathbf{X}$, $\text{rank}(\widehat{\mathbf{X}}) \leq \min\{m, \text{rank}(\mathbf{X})\} \leq m$

2.4. Deep Linear Gradients

TODO

3. Sigmoid Networks

Composition of linear unit w/ non-linear activation function.

Ridge unit: $f = \phi \circ \iota$, $f(\mathbf{x}; \boldsymbol{\theta}) = \phi(\mathbf{x}^\top \boldsymbol{\theta})$

For any loss function ℓ, the parameter updates read

Updates: $\nabla_{\boldsymbol{\theta}}(\ell \circ \phi)(\mathbf{x}^\top \boldsymbol{\theta}) = (\mathcal{L} \circ \phi)'(\mathbf{x}^\top \boldsymbol{\theta})\, \underbrace{\mathbf{x}}_{\text{direction fully determined by } \mathbf{x}\ \hookleftarrow}$

3.1. Threshold units (not differentiable)

Sign unit & Heavyside: $\text{sgn}(\mathbf{x}^\top \boldsymbol{\theta}) \in \{-1, 1\}$, $H(z) \in \{0, 1\}$

The perceptron is a symmetric threshold unit.

Perceptron: $\ell(\mathbf{x}, y; \boldsymbol{\theta}) = \min\{0,\ y\mathbf{x}^\top \boldsymbol{\theta}\} = \max\{0,\ -y\mathbf{x}^\top \boldsymbol{\theta}\}$,

$$\nabla_{\boldsymbol{\theta}} \ell(\mathbf{x}, y; \boldsymbol{\theta}) \equiv \Delta\boldsymbol{\theta} = \begin{cases} \mathbf{0} & y\mathbf{x}^\top \boldsymbol{\theta} \geq 0 \\ y\mathbf{x} & \text{otw.} \end{cases}$$

Given the labels $y = \pm 1$, the perceptron starts at $\boldsymbol{\theta} \leftarrow \mathbf{0}$ and **updates** the weights $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \Delta\boldsymbol{\theta}$ only in case of a mistake.

3.2. Logistic unit (C^∞ smooth & monotone)

Sigmoid: $\sigma(z) = \frac{1}{1 + \mathrm{e}^{-z}} = \frac{\mathrm{e}^z}{\mathrm{e}^z + 1} \in [0, 1]$, $z := \mathbf{x}^\top \boldsymbol{\theta}$

$\sigma'(z) = \sigma(z)(1 - \sigma(z)) = \sigma(z)\sigma(-z)$, $\sigma^{-1}(z) = \ln \frac{z}{1 - z}$

Hyperbolic: $\tanh(z) = \frac{\mathrm{e}^z - \mathrm{e}^{-z}}{\mathrm{e}^z + \mathrm{e}^{-z}} = 2\sigma(2z) - 1 \in [-1, 1]$

$\tanh(z) = \frac{2\sinh(z)}{2\cosh(z)}$, $\nabla_z \tanh(z) = 1 - \tanh^2(z)$

3.2.1. Softmax (k classes)

Softmax: $\sigma_i^{\max}(z_i) = \frac{\exp(z_i)}{\sum_{j=1}^k \exp(z_j)}$, $z_i := \mathbf{x}^\top \boldsymbol{\theta}_i$

We recover the sigmoid for $k = 2$:

soft-max w/ $\boldsymbol{\theta}_1, \boldsymbol{\theta}_2 \iff$ sigmoid w/ $\boldsymbol{\theta} := \boldsymbol{\theta}_1 - \boldsymbol{\theta}_2$

3.2.2. Cross-entropy loss

Probability: $\mathbb{P}(Y = y \mid \mathbf{x}; \theta) = \sigma(yz)$, $z = \mathbf{x}^\top \boldsymbol{\theta}$

Binary CE: $\ell(\mathbf{x}, y; \boldsymbol{\theta}) = -\ln \sigma(y\mathbf{x}^\top \boldsymbol{\theta}))$ for $y \in \{-1, 1\}$

$\ldots = -y \ln \sigma(z) - (1 - y)\ln(1 - \sigma(z))$ for $y \in \{0, 1\}$

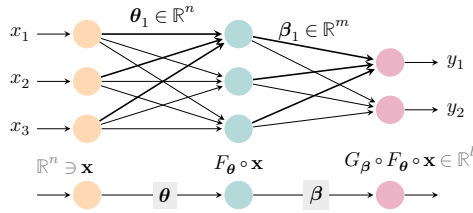
$-\nabla_{\boldsymbol{\theta}} \ell(\mathbf{x}, y; \boldsymbol{\theta}) = \sigma(-yz)y \cdot \mathbf{x}$

Softmax CE: $\ell(\mathbf{x}, \mathbf{y}; \boldsymbol{\Theta}) = -\mathbf{y} \cdot \ln \sigma^{\max}(\mathbf{x}; \boldsymbol{\Theta})$

$= -\sum_{i=1}^k y_i \mathbf{x}^\top \boldsymbol{\theta}_i + \underbrace{\ln(\sum_i \exp(\mathbf{x}^\top \boldsymbol{\theta}_i))}_{\text{norm. constant}}$

$\nabla_{\boldsymbol{\theta}_i} \ell(\mathbf{x}, \mathbf{y}; \boldsymbol{\Theta}) = (\sigma_i^{\max} - y_i) \cdot \mathbf{x}$

3.3. Multilayer perceptron (MLP)



Forward: $y_k \equiv f_k(\mathbf{x}; \boldsymbol{\beta}, \boldsymbol{\theta}) = \sum_{j=1}^m \beta_{kj}\, \sigma(\boldsymbol{\theta}_j^\top \mathbf{x}) = \boldsymbol{\beta}_k^\top \sigma(\boldsymbol{\Theta}\mathbf{x})$

Backw. (SGD): $\theta \leftarrow \theta - \eta \cdot \frac{\partial \frac{1}{2}(f(\mathbf{x}) - y)^2}{\partial \theta}$, $\theta \in \{\beta_j, \theta_{ji}\}$, $\eta > 0$

4. Approximation Theory

4.1. Universality

Weierstrass Theorem: Polynomials $\mathcal{P}$ are **dense** in $C(I)$, *i.e.* set of intervals, where $I = [a, b]$ for any $a < b$.

w.l.o.g.: Assume $I = [0, 1]$ & define $\phi : C(I) \mapsto C([a, b])$, $\phi(f) = f \circ \phi$, $\phi = (1 - t)a + tb$.

An MLP $\mathcal{H}_\sigma^n$ with any continuous, *non*-polynomial activation function $\sigma \in C^\infty(\mathbb{R}^n)$ is a universal approximator.

$\mathcal{G}_\sigma^n = \{g : g(\mathbf{x}) = \sigma(\boldsymbol{\theta}^\top \mathbf{x}) \mid \boldsymbol{\theta} \in \mathbb{R}^n\}$, $\mathcal{H}_\sigma^n = \text{span}(\mathcal{G}_\sigma^n)$

4.2. Complexity

Universality is the rule not the exception but at what cost? Deeper networks may yield better approximations but there's no evidence for/against it.

- *MLPs do not suffer from the curse of dimensionality* when approximating func. g with $C_g := \int \|\omega\| |\hat{g}(\omega)|\, \mathrm{d}\omega < \infty$.
- approx. error rate $\propto 1/m$ (m = #units)

5. Backpropagation

$\mathbf{z}_0 = \mathbf{x}$ ▷ Forward pass
for $l = 1 \dots k$ **do**
  $\mathbf{z}_l = F_l(\mathbf{z}_{l-1})$
end for
$\boldsymbol{\xi}_{k+1} = \partial \ell(\mathbf{z}_k)$ ▷ Backward pass
for $l = k \dots 1$
  $\boldsymbol{\xi}_l = \boldsymbol{\xi}_{l+1} \cdot \partial F_l(\mathbf{z}_{l-1})$
end for
for $l = 1 \dots k$ **do** ▷ Local computations
  $\nabla_l = \boldsymbol{\xi}_{l+1} \cdot F_l'(\mathbf{z}_{l-1})$
end for

5.1. Layers

DAG: Directed Acyclic Graphs represent connectivity structure between computational units.

In DNNs typically units are arranged in sequence of **layers**

Composition: $F = F_{k:1} = F_k \circ \cdots \circ F_1$

$\text{width}_l \equiv n_l := \dim(\text{range}(F_l))$, $\#\text{units} = \sum_{l=1}^k n_l$

Such layer-to-layer maps successively tansform input into intermediate representations

Activation: $\mathbf{z}_l := F_{l:1}(\mathbf{x}) = (F_l \circ F_{l-1:1})(\mathbf{x}) = F_l(\mathbf{z}_{l-1})$

$\mathbf{z}_0 := \mathbf{x}$: input layer, $\mathbf{y} =: \mathbf{z}_k$: output layer

5.2. Multivariate analysis

Chain rule: $\partial(g \circ f) = (\partial g \circ f) \cdot f$ generalized to maps

$\partial F_{g:f} \equiv \partial(F_g \circ F_f) = (\partial F_g \circ F_f) \cdot \partial F_f$

for DNNs: $\partial F_{l:1} = \prod_{l=k}^1 \partial F_l \circ F_{l-1:1}$

Notation: $F\colon \mathbb{R}^n \times \mathbb{R}^d \to \mathbb{R}^m$, inputs $\mathbf{x} \in \mathbb{R}^n$, parameters $\boldsymbol{\theta} \in \mathbb{R}^d$

$$\partial f = \left(\frac{\partial f}{\partial x_i}\right)_i \overset{m=1}{=} \partial F = \left(\frac{\partial F_j}{\partial x_i}\right)_{ji}, \quad f' = \nabla_\theta f, \quad F' = \nabla_\theta F$$

5.3. Gradient descent

For learning we're not interested in prediction F, but in loss f

Loss: $f := \ell \circ F = \ell \circ F_{k:1} = \ell(F(\mathbf{x}, \boldsymbol{\theta}), \mathbf{y})$, $\ell : \mathcal{Y} \times \mathcal{Y} \to \mathbb{R}$

The (normalized) **negative gradient** $\propto -\nabla_\theta f(\mathbf{x}, \boldsymbol{\theta})$ will define the direction of the steepest descent on the loss.

⚠: There exist two different gradients w.r.t. maps F_l, *input/activity* and ***parameters***. Don't be confused!

Gradient: $\nabla_{\boldsymbol{\theta}_l} F = (\partial F_{k:l+1} \circ F_{l:1}) \cdot (F_l' \circ F_{l-1:1})$ $(\forall l)$

$$\nabla_{\boldsymbol{\theta}_l} f(\mathbf{x}) = \underbrace{\partial(\ell \circ F_{k:l+1})(\mathbf{z}_l)}_{\boldsymbol{\xi}_{k+1}\text{: downstream}} \cdot \underbrace{F_l'(\mathbf{z}_{l-1})}_{\text{local Jacob. } (\mathbf{z}_{l-1}\text{: upstream})}$$

vector-Jacobian product: for $\mathbf{z} = \mathbf{W}\mathbf{x}$ we find $\frac{\partial \mathbf{z}}{\partial \mathbf{W}} = \mathbf{x}^\top$

5.4. Parameter gradients

Ridge function layer: $\nabla_{\boldsymbol{\theta}_{lj}} \ell(\mathbf{x}) = \underbrace{\xi_{lj}}_{\text{Scalar}} \cdot \underbrace{\sigma'(\boldsymbol{\theta}_{lj} \cdot \mathbf{z}_{l-1})}_{\text{Sensitivity of unit}} \cdot \underbrace{\mathbf{z}_{l-1}}_{\text{Direction}}$

Deep lin. netw.: $F(\mathbf{x}) = \prod_{l=k}^1 \boldsymbol{\theta}_l \mathbf{x}$, $f = \ell \circ F$

$\partial F_l = \boldsymbol{\theta}_l$, $\partial F_{k:l} = \prod_{r=k}^l \boldsymbol{\theta}_r$, $\boldsymbol{\xi}_k = \nabla \ell(\mathbf{z}_k)$, $\boldsymbol{\xi}_{l-1} = \boldsymbol{\xi}_l \boldsymbol{\theta}_l$

$\nabla_{\boldsymbol{\theta}_l} f(\mathbf{x}) = \begin{bmatrix}\boldsymbol{\theta}_k & \dots & \boldsymbol{\theta}_{l+1} & \delta\end{bmatrix} \begin{bmatrix}\boldsymbol{\theta}_{l-1} & \dots & \boldsymbol{\theta}_1 & \mathbf{x}\end{bmatrix}^\top = \boldsymbol{\xi}_l \mathbf{z}_{l-1}^\top$

6. Rectified Networks

6.1. Rectified linear unit — ReLU

ReLU: $(\mathbf{x}, \boldsymbol{\theta})_+ := \max\{0, \mathbf{x} \cdot \boldsymbol{\theta}\}$, $\partial(z)_+ = \begin{cases} 1 & \text{if } z > 0 \\ 0 & \text{if } z < 0 \end{cases}$

- backprop very simple: gradient either 0 or the weight value
- sparseness of Jacobi matrix can be exploited

$$\partial F_l = \tilde{\boldsymbol{\Theta}} := \left(\tilde{\boldsymbol{\theta}}_{li}^\top\right)_i, \quad \tilde{\boldsymbol{\theta}}_{ij} = \begin{cases} \mathbf{0} & \text{if } z_{lj} = 0 \\ \boldsymbol{\theta}_{lj} & \text{otw.} \end{cases} \leftarrow \text{weight value!}$$

- can lead to dying *(inactive)* ReLU

6.2. Absolute value unit — AbsU

AbsU: $\partial|z| = \begin{cases} 1 & z > 0 \\ -1 & z < 0 \end{cases}$, $(z)_+ = \frac{z+|z|}{2}$ and $|z| = 2(z)_+ - z$

6.3. Smooth ReLU approximations

Softplus: $\sigma^{\text{softp}}(z) = \log(1 + \mathrm{e}^z)$

Exp. LU: $\sigma^{\text{expLU}}(z) = \begin{cases} z & z \geq 0 \\ \mathrm{e}^z - 1 & \text{otw.} \end{cases}$

Leaky ReLU: $\sigma^{\ell\text{ReLU}}(z) = \begin{cases} z & z \geq 0 \\ \epsilon \cdot z & \text{otw.} \end{cases}$

6.4. Piecewise linear approximation

Theorem: Piecewise linear functions are *dense* in $C([0;1])$.

with m pieces: $g(x) = \underbrace{ax + b}_{\text{linear}} + \underbrace{\sum_{i=1}^{m-1} c_i (x - x_i)_+}_{\text{changes of slope, but } no \text{ shift}}$

6.5. Hinge functions

2+ hyperplanes glued together: $g(\mathbf{x}) = \max(\boldsymbol{\theta}_1 \mathbf{x} + b_1, \boldsymbol{\theta}_2 \mathbf{x} + b_2)$

6.6. Polyhedral functions

Polyhedral: f is *polyhedral* $\iff$ epi(f) is a polyhedral set

Epigraph: $\text{epi(f)} := \{(\mathbf{x}, t) \in \mathbb{R}^{n+1} \colon f(\mathbf{x}) \leq t\}$

A *set* S is polyhedral $\iff S = \{\mathbf{x} \in \mathbb{R}^n \colon \boldsymbol{\theta}_j \mathbf{x} + b_j \geq 0, j = 1, \dots, r\}$

$\to$ finite #hyperplanes sufficient to approximate a polyh. set

Thm.: Every continuous pieccewise linear function f can be written as the difference of **two** polyhedral functions.

$$f(\mathbf{x}) = \max_{(\boldsymbol{\theta}, b) \in \mathcal{A}^+} \{\boldsymbol{\theta}\mathbf{x} + b\} - \max_{(\mathbf{w}, b) \in \mathcal{A}^-} \{\mathbf{w}\mathbf{x} + b\}$$

7. Optimization

7.1. Objectives

Squared loss: $\ell_{\mathbf{y}}(\boldsymbol{\nu}) = \frac{1}{2}\|\mathbf{y} - \boldsymbol{\nu}^2\|$

Classification loss: $\ell_y(\nu) = \begin{cases} 0 & \text{if } \nu = y \\ 1 & \text{otw.} \end{cases}$ (has no gradients)

Probabilistic loss: $\ell_{\mathbf{y}}(\boldsymbol{\nu}) = -\log p(\mathbf{y}; \boldsymbol{\nu})$

- **Gaussian models:** $p(\mathbf{y}; \boldsymbol{\nu}) = \prod_{j=1}^m \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{1}{2}(y_j - \nu_j)\right)^2$
  $\implies \ell_{\mathbf{y}}(\boldsymbol{\nu}) = \frac{1}{2}\|\mathbf{y} - \boldsymbol{\nu}\|^2 + \text{const}$
- **Exponential:** $p(\mathbf{y}; \boldsymbol{\nu}) = h(\mathbf{y}) \exp(\mathbf{y}\boldsymbol{\nu} - \phi(\boldsymbol{\nu}))$
  $\implies \ell_{\mathbf{y}}(\boldsymbol{\nu}) = -\mathbf{y}\boldsymbol{\nu} + \phi(\boldsymbol{\nu}) + \text{const}$
- **Poisson models:** $p(y; \nu) = \nu^y \frac{\mathrm{e}^{-\nu}}{y!}$
  $\implies \ell_y(\nu) = \nu - y \log \nu + \text{const}, \ \nu > 0$

7.2. Expected vs. empirical risk

Expected: $\mathcal{R}(\boldsymbol{\theta}) = \mathbb{E}_{(\mathbf{x},\mathbf{y})}[\ell(F(\mathbf{x}; \boldsymbol{\theta}))] \implies \boldsymbol{\theta}^* = \arg\min_{\boldsymbol{\theta}} \mathcal{R}(\boldsymbol{\theta})$

Empirical: $\hat{\mathcal{R}}(\boldsymbol{\theta}) = \frac{1}{s} \sum_{t=1}^s \ell_{\mathbf{y}^t}(F(\mathbf{x}^t; \boldsymbol{\theta}))$

7.3. Gradient descent — GD

Use Taylor approx. $f(\mathbf{x} + \Delta\mathbf{x}) \approx g(\mathbf{x})$ and set $\nabla g(x) \overset{!}{=} 0$.

$\implies$ **Newton's method:** $\Delta\mathbf{x} = -[\nabla^2 f(\mathbf{x})]^{-1} \nabla f(\mathbf{x})$

Idea: Find matrix $\mathbf{B}$ close to the Hessian, i.e. $\|\nabla^2 f(\mathbf{x}) - \mathbf{B}\|_2 \leq \epsilon$

Gradient descent: Assuming $\nabla^2 f(\mathbf{x}) = \frac{1}{\eta}\mathbf{I}$, we recover

$\Delta\mathbf{x} = -\eta \nabla f(\mathbf{x})$, step size $\eta > 0$

Gradient flow: $\dot{\mathbf{x}} = -\nabla f(\mathbf{x})$, grad. descent with $\eta \to 0$

7.3.1. Optimal learning rate

Start w/ convex quadratic objective $f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^\top \mathbf{Q}\mathbf{x} - \mathbf{q}\mathbf{x}$ with $\mathbf{Q} = \mathbf{U}\boldsymbol{\Lambda}\mathbf{U}^\top$ p.s.d., thus f convex. Change the basis as $\mathbf{x} \leftarrow \mathbf{U}^\top$ and $\mathbf{q} \leftarrow \mathbf{U}^\top \mathbf{q}$ and effectively minimize the *separable* problem: $f(\mathbf{x}) = \sum_{i=1}^n g_i(x_i)$, $g_i(z) = \frac{\lambda_i}{2} z^2 - q_i z$, $\lambda_i > 0$.

Exp. convergence if: $\eta < \frac{2}{\lambda}$ (1-dim.), $\eta \leq \frac{2}{\lambda_{\max}(\mathbf{Q})}$ (n-dim.)

Optimal: $\eta^* = \min_\eta \max\{\eta\lambda_{\max} - 1, 1 - \eta\lambda_{\min}\} = \frac{2}{\lambda_{\max} + \lambda_{\min}}$

7.3.2. Convexity

L-smooth function: $\|\nabla f(\mathbf{x}) - \nabla f(\mathbf{x} + \Delta\mathbf{x})\| \leq L\|\Delta\mathbf{x}\|$

$C^2 \rightsquigarrow f(\mathbf{x} + \Delta\mathbf{x}) \leq f(\mathbf{x}) + \nabla f(\mathbf{x}) \cdot \Delta\mathbf{x} + \frac{L}{2}\|\Delta\mathbf{x}\|^2$

μ-strongly convex: $f(\mathbf{x} + \Delta\mathbf{x}) \geq f(\mathbf{x}) + \nabla f(\mathbf{x}) \cdot \Delta\mathbf{x} + \frac{\mu}{2}\|\Delta\mathbf{x}\|^2$

Convex if $\mu = 0$.

$\implies$ Can be sandwiched: $\mu\mathbf{I} \preceq \nabla^2 f \preceq L\mathbf{I}$

TODO: Lec. ch08-a: slide 26-39

Convexity: $\ell(\lambda w + (1-\lambda)w') \le \lambda\ell(w) + (1-\lambda)\ell(w')$
$\iff \ell(w) \ge \ell(w') + \nabla\ell(w')^\top(w-w')$
Given also ***L*-smoothness**, i.e. $\|\nabla\ell(w) - \nabla\ell(w')\| \le L\|w-w'\|$, then $\implies \ell(w) \le \ell(w') + \nabla\ell(w')^\top(w-w') + \frac{L}{2}\|w-w'\|^2$.

7.4. Stochastic gradient descent — SGD

Approximate true gradient with mean gradient of batch. Sacrifice statistical efficiency at benefit of reduced variance.

Unbiased: $\mathbb{E}[\nabla f_I(\mathbf{x})] = \sum_{i=1}^n \frac{1}{n}\nabla f_i(\mathbf{x}) = \nabla f(\mathbf{x})$

Variance: $\mathbb{V}[\mathbf{x}] = \frac{1}{n}\sum_{i=1}^n \|\nabla f(\mathbf{x}) - \nabla f_i(\mathbf{x}^2)\|$, $\nabla f(\mathbf{x}^*) = 0$

Note: Smaller batch sizes introduce more noise, yet usually give better results in the non-convex case

7.4.1. Learning rates

In theory: $\eta_k \propto 1/k$, $\sum_{k=1}^\infty \eta_k = \infty$, $\sum_{k=1}^\infty \eta_k^2 < \infty$

in practice: trial and error (e.g. may use $\eta_k = \mathrm{e}^{-k}$)

7.4.2. SVRG

$$\mathbf{x}^{k+1} = \mathbf{x}^k - \eta\underbrace{[\nabla f_i(\mathbf{x}^k) - \nabla f_i(\bar{\mathbf{x}}) + \nabla f(\bar{\mathbf{x}})]}_{\mathbb{E}[\cdot]=\nabla f(\mathbf{x}),\quad \mathbb{V}[\cdot]<\mathbb{V}[\nabla f_i(\mathbf{x})]}$$

Expensive but do only once in a while. Very effective in convex optimization (**not** so for DNNs).

7.5. Momentum and adaptivity

7.5.1. Nesterov's accelerated method (speed-up conv.)

(1) Extrapolate by momentum: $\mathbf{y}_{k+1} = \mathbf{x}_k + \beta(\mathbf{x}_k - \mathbf{x}_{k-1})$,
(2) then do gradient descent: $\mathbf{x}_{k+1} = \mathbf{y}_{k+1} - \eta\nabla f(\mathbf{y}_{k+1})$.
$\beta = 0$ is standard gradient descent.

7.5.2. Heavy ball method ("shoot more straight")

Same as Nesterov but do gradient at $\mathbf{x}^k$ and then add momentum/extrapolate: $\mathbf{x}^{k+1} = \mathbf{x}^k - \eta\nabla f(\mathbf{x}^k) + \beta(\mathbf{x}^k - \mathbf{x}^{k-1})$.

7.5.3. AdaGrad

$\mathbf{x}^{k+1} = \mathbf{x}^k - \eta\mathbf{\Lambda}^k\nabla f(\mathbf{x}^k)$, where $\mathbf{\Lambda}^k = \mathrm{diag}(\lambda_i^k)$, $\lambda_i^k = \frac{1}{\sqrt{\gamma_i^k + \delta}}$, regularizer $\delta > 0$. Define: $\boldsymbol{\gamma}^k = \boldsymbol{\gamma}^{k-1} + \nabla f(\mathbf{x}^k) \odot \nabla f(\mathbf{x}^k)$.

7.5.4. Adam (combines momentum & adaptivity)

Momentum: $\mathbf{m}^k = \beta\mathbf{m}^{k-1} + (1-\beta)\nabla f(\mathbf{x}^k)$, $\beta \in [0;1]$, $\mathbf{m}^0 = \mathbf{0}$, $\mathbf{x}^{k+1} = \mathbf{x}^k - \frac{\eta}{1-\beta^k}\mathbf{m}^k$

Adaptivity: $\boldsymbol{\gamma}^k = \alpha\boldsymbol{\gamma}^{k-1} + (1-\alpha)[\nabla f(\mathbf{x}^k) \odot \nabla f(\mathbf{x}^k)]$, $\mathbf{x}^{k+1} = \mathbf{x}^k - \frac{\eta}{1-\beta^k}\mathbf{\Lambda}^k\mathbf{m}^k$, where $\mathbf{\Lambda}^k = \mathrm{diag}(\lambda_i^k)$, $\frac{1}{\lambda_i^k} = \sqrt{\frac{\gamma_i^k}{1-\alpha^k}} + \delta$

7.5.5. Compressed stochastic gradients

Sending information across nodes may be expensive. Idea: use only sgn of the update. $\mathbf{x}^{k+1} = \mathbf{x}^k - \eta_k \mathrm{sgn}\big(\nabla f_{I(k)}(\mathbf{x}^k)\big)$

TODO: Lec. ch08-b: Rand. feat. regr. and neural tangent kernel

8. Convolutional Networks

Fourier Trsf.: $(\mathcal{F}(f))(u) := \int_{\mathbb{R}} \mathrm{e}^{-2\pi i t u} f(t)\,\mathrm{d}t$

Convolution: $(f*h)(u) := \int_{\mathbb{R}} f(t)h(u-t)\,\mathrm{d}t = \int_{\mathbb{R}} f(u-t)h(t)\,\mathrm{d}t$
shift equiv.: $f_\triangle * h = (f*h)_\triangle$, $f_\triangle(t) := f(t+\triangle)$

8.1. Convolutional architectures

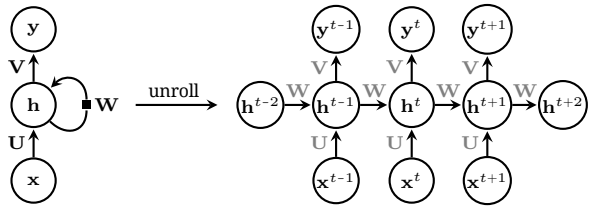
Pyramidal architecture: increase channels and reduce resolution w/ increasing depth; AlexNet, VGG, Inception

9. Recurrent Neural Networks

Linear RNN: $\mathbf{h}^t = F(\mathbf{h}^{t-1}, \mathbf{x}^t; \boldsymbol{\theta})$, $\mathbf{h}^0 = \mathbf{0}$, $F = \sigma \circ \bar{F}$
$\bar{F}(\mathbf{h}, \mathbf{x}; \boldsymbol{\theta}) = \mathbf{W}\mathbf{h} + \mathbf{U}\mathbf{x} + \mathbf{b}$, $\boldsymbol{\theta} = (\mathbf{U}, \mathbf{W}, \mathbf{b}, \ldots)$

optional: $\mathbf{y} = H(\mathbf{h}; \boldsymbol{\theta})$, $H(\mathbf{h}; \boldsymbol{\theta}) := \sigma(\mathbf{V}\mathbf{h} + \mathbf{c})$, $\boldsymbol{\theta} = (\ldots, \mathbf{V}, \mathbf{c})$

You can loot at is as a neural network. However, the first input has N layers, the second N - 1, etc., and the last has two layers.



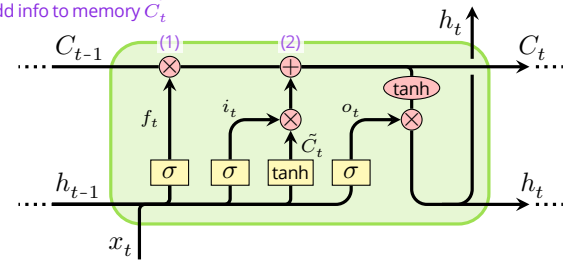
Backpropagation through time:

- Propagate derivates backwards through time
- Parameter sharing leads to a sum over t
- *Def.* shortcut $\dot{o}_i^t := \sigma'(\bar{F}_i(h^{t-1}, x^t))$, $\mathbf{S}(\mathbf{h}^r) = \mathrm{diag}(\dot{o}_1^r, \ldots, \dot{o}_n^r)$
- **RNN gradients:** $\nabla_{\mathbf{x}^t}\mathcal{R} = \left[\prod_{r=t+1}^s \mathbf{W}^\top \mathbf{S}(\mathbf{h}^r)\right] \cdot \mathbf{J}_H \cdot \nabla_{\mathbf{y}}\mathcal{R}$

9.1. Memory Units – LSTM

Classical RNNs easily forget (info gets "washed out").

(1): element-wise, to (not) pass information,
(2): add info to memory C_t



- Information propagates along the chain
- can flow unchanged, only selectively changed by σ-gates
- can forget (when $f_t \approx 0$)
- LSTMs and GRUs can learn active memory strategies
- difficult to understand and resource-hungry

9.2. Seq2Seq learning

Encoder RNN $\mathbf{x} \mapsto \mathbf{z}$ and **Decoder RNN** $\mathbf{z} \mapsto \mathbf{y}^{1:S}$ e.g. use encoder output as input for decoder and stop symbol <EOS>.
Can use Seq2Seq also for **image-to-text** sequencing, ...!

Downsides of RNN: (i) they don't work well for longer sequences, (ii) cannot be parallelized since architecture is sequential.

10. Deep Gradients

10.1. Short connectivity

Residual connections: shortcut layers and **add** back in, $g(\mathbf{x}) = f(\mathbf{x};\boldsymbol{\theta}) + \mathbf{x}$, or if dimensions don't match: $f(\mathbf{x};\boldsymbol{\theta}) + \mathbf{W}\mathbf{x}$

Skip connections: shortcut layers and **concatenate** back in. Connect layer output to all downstream layers.

10.2. Normalization

We observe **vanishing gradients**, i.e. poor backpropagation of errors, when incoming data to activation function is not normalized.

10.2.1. Batch normalization $\implies$ across many data points

- Learning rate η is otherwise very sensitive
- Key idea: normalize the layer activation (and backpropagate through normalization!)

(+) very effective, broadly used in vision
(−) dependence on batch size, not suitable for all architectures

10.2.2. Layer normalization $\implies$ across units in same layer

- In scenarios like recurrent NNs, we have batch size = 1; batch normalization is not possible.
- Use population average of activations in layer as reference

(+) very effective, used in NLP (transformers, RNNs)
(−) no consistent benefits

11. Regularization

Weight decay: L2-regularization; pulls weights towards zero.

Early stopping: use validation data to estimate expected risk.
$\rightarrow$ acts as approximate L2-regularizer!

Bagging: combine models trained on bootstrap samples S_r^k.
$\rightarrow$ almost always beneficial, but expensive!

Dropout: randomly "drop" subsets of units in network.
$\rightarrow$ each neuron learns to detect a feature that is *generally* helpful! | $\rightarrow$ effectively defines an exponential **ensemble** w/o suffering the drawback of having to evaluate multiple layouts.

11.1. Data augmentation

Exploit input **invariances** by generating virtual examples subject to transformations τ (highly domain specific).

Injecting noise: to inputs, weights and/or targets

11.2. Task augmentation

Pre-training: use pre-trained parts of the model. May also *fine-tune* parameters.

Semi-superivsed learning: optimize *additive combination* of supervised and unsupervised risk, while **sharing parameters**. → (typically) when more unlabeled that labeled data.

Multitask learning: share (low level) representations across tasks and learn jointly. → Learn high level repr. *per task*.

12. Theory of DNNs

Consistency: **VC inequality:** if worst-case empirical risk minimizer diverges, it will happen for the true case.

Spectral complexity: you can see the randomness of your data given the normalized margins!

$$\text{Test error}(f) \le \text{empirical } \gamma\text{-margin error}(f)$$

Double descent risk curves: DNNs contradict the classical view of bias-variance tradeoff. *Note:* this behavior is invisible when we use regularizers from the beginning.

13. Attention

RNNs: learn to encode information relevant for the future.

Attention: select what is relevant in retrospective.

13.1. Self attention ($\mathbf{h}_e^j$: encoder hidden state j)

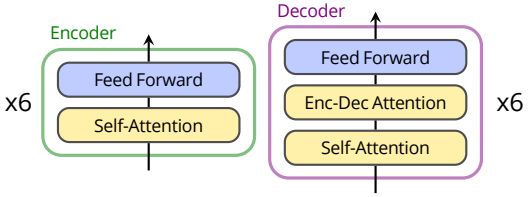
Softmax Gating: $f_\phi(\boldsymbol{\xi}, \mathbf{h}_e^{1:s}) = \frac{1}{\sum_j \exp(\phi(\boldsymbol{\xi}, \mathbf{h}_e^1))} \begin{pmatrix} \exp(\phi(\boldsymbol{\xi}, \mathbf{h}_e^1)) \\ \vdots \end{pmatrix} \in \mathbb{R}^s$

Gating function: Similarity or compatibility $\phi\colon \mathbb{R}^n \times \mathbb{R}^m \to \mathbb{R}$.
→ often an MLP, e.g. $\phi(\boldsymbol{\xi}, \mathbf{h}_3) = \boldsymbol{\xi} \cdot \mathbf{h}_e$ for $n{=}m$

Self-gated Attention: $\mathbf{z}^{1:n} = [\mathbf{h}_e^1, \dots, \mathbf{h}_e^s] \cdot f_\phi(\boldsymbol{\xi}, \mathbf{h}_e^{1:s}) \in \mathbb{R}^{n \times s} \times \mathbb{R}^s$
→ use those "read-outs" as input to the decoding RNN!

13.2. Transformers

Sequential nature of RNNs lead to *memory limitations* and increased *computation time*. IDEA: Preserve attention, and encoder-decoder architecture, but...



Self attention: captures relations between words in a sentence. Use (query-key-value) word embeddings. → think of it as a Google search / retrieval process → can be computed in parallel! Afterwards, use layer normalization.

Enc-Dec attention: mimics attention mechanism of Seq2Seq models → input to output attention!

(+) very powerful for solving encoder-decoder NLP problems

(−) hard to train, attention results are not well interpretable

13.2.1. Embedding from language models — ELMO

*Bi*directional LSTM! l2r for prediction, r2l for context.

13.2.2. Bidirect. Enc. Repr. from Transformers — BERT

Builds from *ELMO* and *GPT* but is a **few-shot learner**. Compared to traditional *fine-tuning*, this shows that such large NLP models have **learnt to learn**.

14. Variational Autoencoders

Define: $\xrightarrow[\text{prior}]{p_{\theta'}(z)} \mathcal{Z} \xrightarrow[\text{likelihood}]{\text{dec}_\theta(z)=p_\theta(x|z)} \mathcal{X} \xrightarrow[\text{approx. posterior}]{\text{enc}_\phi(x)=q_\phi(z|x)} \mathcal{Z}$
→ sample/observe $\mathcal{X}_i$ from latent representation $\mathcal{Z}$

Train: $\max_{\theta',\theta,\phi} \sum_i \underbrace{\log p_{\theta',\theta}(x_i)}_{(*)\text{ indep. of } Z}$

$$(*) = \mathbb{E}_{Z\sim q_\phi(\cdot|x_i)}\left[\log\left(\frac{p_{\theta',\theta}(x_i,Z)}{p_{\theta','theta}(Z|x_i)}\frac{q_\phi(Z|x_i)}{q_\phi(Z|x_i)}\right)\right]$$

$\mathcal{L}(x_i,\theta,\phi) \equiv$ **ELBO** = Infomax − Regularisation term

$$= \overbrace{\mathbb{E}[\log p_\theta(x_i \mid Z)] - D_{\mathrm{KL}}\big(q_\phi(\cdot \mid x_i) \parallel p_{\theta'}(\cdot)\big)}$$
$$+ D_{\mathrm{KL}}\big(q_\phi(\cdot \mid x_i) \parallel p_{\theta',\theta}(\cdot \mid x_i)\big) \ge \mathcal{L}(x_i,\theta,\phi)$$

$$\implies \theta^*, \phi^* = \arg\max_{\theta,\phi} \mathcal{L}(x_i,\theta,\phi)$$

Requirements for good representation:

- **informative:** $\theta^* = \arg\max_\theta I(X;Z) = \arg\max_\theta \mathbb{E}_{X,Z}[\log p(X \mid Z)] - const_{w.r.t.\,\theta} \approx \arg\max_\theta \sum_i \mathbb{E}_{Z|X}[\log p(x_i \mid Z)]$
- **disentangled:** components in $\mathcal{Z}$ associated with distinct feature in $\mathcal{X}$ (see D_{KL} in ELBO).
- **robust:** noise in $\mathcal{Z}$ doesn't substantially affect $\mathcal{X}$ (and vice versa) → choice of approx. posterior!

15. Adversarial Robustness

How to measure robustness of linear, **binary** classifier? $\arg\min_r \|r\|_p$ s.t. $f_1(x+r) - f_2(x+r) < 0$.

For $p=2$: $r_i = \frac{f_1(x)-f_2(x)}{\|w_2-w_1\|}(w_2-w_1)$. **Multi-class** case: $\arg\min_{\{r_i\}} \|r_i\|_2$ ← pick min. distance from x to any polytope.

IDEA: Improve robustness through adversarial training!

Standard train loss: $\min_\theta \ell(f_\theta(x), y)$

L_p-robust train loss: $\min_\theta \max_{\|r\|_p \le \epsilon} \ell(f_\theta(x+r), y)$

Project gradient **ascent** via $r \mapsto \begin{cases} r_i & |r_i| \le \epsilon \\ \epsilon & \text{otw.} \end{cases} \le \epsilon \cdot \operatorname{sgn}(r_i)$

Appendix

Taylor: $f(\mathbf{x}+\Delta\mathbf{x}) \approx f(\mathbf{x}) + \nabla f(\mathbf{x})^\top \Delta\mathbf{x} + \frac{1}{2}\Delta\mathbf{x}^\top \nabla^2 f(\mathbf{x}) \Delta\mathbf{x}$

Cauchy-Schwarz: $|\mathbb{E}[X,Y]|^2 \le \mathbb{E}[X^2]\,\mathbb{E}[Y^2]$

Jensen, $f(\cdot)$ convex: $f(\mathbb{E}[x]) \le \mathbb{E}[f(X)]$

Derivative: $\nabla_{\mathbf{v}} f(\mathbf{w}) = \nabla f(\mathbf{w})^\top \mathbf{v} = \lim_{\lambda\to 0} \frac{f(\mathbf{w}+\lambda\mathbf{v}) - f(\mathbf{w})}{\lambda}$

Probability / statistics

- Gaussian: $\mathcal{N}(x \mid \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{1}{2}\frac{(x-\mu)^2}{\sigma^2}\right)$
  $\mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}, \boldsymbol{\Sigma}) = \frac{1}{\sqrt{(2\pi)^d |\boldsymbol{\Sigma}|}} \exp\left(-\frac{1}{2}(\mathbf{x}-\mu)^\top \boldsymbol{\Sigma}^{-1}(\mathbf{x}-\mu)\right)$
  $X \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma}),\ Y = A + BX \implies Y \sim \mathcal{N}(A + B\boldsymbol{\mu},\ B\boldsymbol{\Sigma}B^\top)$
- Binom.: $f(k,n;p) = \mathbb{P}(X=k) = \binom{n}{k} p^k (1-p)^{n-k}$
- $\mathbb{V}[X] = \mathbb{E}[(X - \mathbb{E}[X])^2] = \mathbb{E}[X^2] - \mathbb{E}[X]^2$
  $\mathbb{V}[X+Y] = \mathbb{V}[X] + \mathbb{V}[Y] + 2\,\mathbb{C}\mathrm{ov}(X,Y)$
- $\mathbb{C}\mathrm{ov}(X,Y) = \mathbb{E}[(X - \mathbb{E}[X])(Y - \mathbb{E}[Y])] = \mathbb{E}[XY] - \mathbb{E}[X]\,\mathbb{E}[Y]$
  $\mathbb{C}\mathrm{ov}(aX, bY) = ab\,\mathbb{C}\mathrm{ov}(X,Y)$

Complete the square:

If $p(\mathbf{x}) \propto \exp(-\frac{1}{2}\mathbf{x}^\top \mathbf{A}\mathbf{x} + \mathbf{x}^\top \mathbf{b})$, then $p(\mathbf{x}) = \mathcal{N}(\mathbf{x} \mid \mathbf{A}^{-1}\mathbf{b}, \mathbf{A}^{-1})$

Conditional Gaussians: $\implies Y|X \sim \mathcal{N}(\tilde{\boldsymbol{\mu}}, \tilde{\boldsymbol{\Sigma}})$, where
$\tilde{\boldsymbol{\mu}} = \boldsymbol{\mu}_Y + \boldsymbol{\Sigma}_{YX}\boldsymbol{\Sigma}_{XX}^{-1}(X - \boldsymbol{\mu}_X)$, $\tilde{\boldsymbol{\Sigma}} = \boldsymbol{\Sigma}_{YY} - \boldsymbol{\Sigma}_{YX}\boldsymbol{\Sigma}_{XX}^{-1}\boldsymbol{\Sigma}_{XY}$)

$$P_{X,Y} = \begin{bmatrix} \mathbf{X} \\ \mathbf{Y} \end{bmatrix} \sim \mathcal{N}\left(\begin{bmatrix} \boldsymbol{\mu}_X \\ \boldsymbol{\mu}_Y \end{bmatrix}, \begin{bmatrix} \boldsymbol{\Sigma}_{XX} & \boldsymbol{\Sigma}_{XY} \\ \boldsymbol{\Sigma}_{YX} & \boldsymbol{\Sigma}_{YY} \end{bmatrix}\right),\ \boldsymbol{\Sigma}_{ij} \text{ p.s.d.}$$

Calculus

- $\int_a^b uv'\,\mathrm{d}x = uv|_a^b - \int_a^b u'v\,\mathrm{d}x$
- $\frac{\partial}{\partial x}\frac{g}{h} = \frac{g'h}{h^2} - \frac{gh'}{h^2}$
- $\frac{\partial}{\partial \mathbf{x}} f(\mathbf{x}) = \mathrm{diag}(f'(\mathbf{x}))$
- $\frac{\partial}{\partial \mathbf{x}}(\mathbf{b}^\top \mathbf{x}) = \frac{\partial}{\partial \mathbf{x}}(\mathbf{x}^\top \mathbf{b}) = \mathbf{b}$
- $\frac{\partial}{\partial \mathbf{x}}(\mathbf{b}^\top \mathbf{A}\mathbf{x}) = \mathbf{A}^\top \mathbf{b}$
- $\frac{\partial}{\partial \mathbf{x}}(\mathbf{x}^\top \mathbf{A}\mathbf{x}) = (\mathbf{A}^\top + \mathbf{A})\mathbf{x} \overset{*}{=} 2\mathbf{A}\mathbf{x}$
  $(*)$: $\mathbf{A}$ sym.
- $\frac{\partial}{\partial \mathbf{X}}(\mathbf{c}^\top \mathbf{X}\mathbf{b}) = \mathbf{c}\mathbf{b}^\top$
- $\frac{\partial}{\partial \mathbf{X}}(\mathbf{c}^\top \mathbf{X}^\top \mathbf{b}) = \mathbf{b}\mathbf{c}^\top$
- Trace trick: $\mathbf{x}^\top \mathbf{A}\mathbf{x} \overset{*}{=} Tr(\mathbf{x}^\top \mathbf{A}\mathbf{x}) \overset{**}{=} Tr(\mathbf{x}\mathbf{x}^\top \mathbf{A}) = Tr(\mathbf{A}\mathbf{x}\mathbf{x}^\top)$
  $(*)$: inner prod., $(**)$: cyclic permut.
- $\frac{\partial}{\partial \mathbf{X}} Tr(\mathbf{X}^\top \mathbf{A}) = \mathbf{A}$
- $|\mathbf{X}^{-1}| = |\mathbf{X}|^{-1}$
- $\frac{\partial}{\partial \mathbf{X}} \log|\mathbf{X}| = \mathbf{X}^{-\top}$
- $\frac{\mathrm{d}}{\mathrm{d}x}|x| = \frac{x}{|x|}$
- $\frac{\partial}{\partial \mathbf{x}}\|\mathbf{x}\|_2 = \frac{\partial}{\partial \mathbf{x}}(\mathbf{x}^\top \mathbf{x}) = 2\mathbf{x}$
- $\frac{\partial}{\partial \mathbf{x}}\|\mathbf{x}-\mathbf{b}\|_2 = \frac{\mathbf{x}-\mathbf{b}}{\|\mathbf{x}-\mathbf{b}\|_2}$
- $\frac{\partial}{\partial \mathbf{x}}\|\mathbf{x}\|_1 = \operatorname{sgn}(\mathbf{x})$ $\operatorname{sgn}(\mathbf{x}) \in \{\pm 1\}^p$ is row-wise